

nRF Connect SDK - 2.9.2

Contents

Zigbee FOTA..... 3

1. Zigbee FOTA

Note

Support for Zigbee R22 has been deprecated in nRF Connect SDK v2.8.0 and it will be removed in one of the future releases, following the [deprecation policy](#). As a result, the nRF52833, nRF52840, and nRF5340 SoCs are not recommended for new Zigbee designs. Experimental support for Zigbee R23 is available for the nRF54L Series as an [nRF Connect SDK Add-on](#).

The Zigbee firmware over-the-air (Zigbee FOTA) library provides Zigbee endpoint definition, which implements clusters responsible for transferring a firmware file through the Zigbee network. The received data is passed as an upgrade candidate through the [DFU multi-image](#) library API to the [DFU target](#).

The library uses the endpoint to:

- Discover Zigbee OTA server by sending Match Descriptor Request broadcast messages.
- Query the discovered server periodically to check if there is a suitable image for the update.
- Transfer the new firmware using the ZCL OTA cluster commands.
- Allow the OTA server to manage the upgrade process using the ZCL OTA control commands.

When used, the Zigbee FOTA library replaces the [FOTA download](#) library in nRF Connect SDK.

Note

The Zigbee FOTA functionality is available for the [Zigbee: Light switch](#) sample.

OTA upgrade entities

The following entities participate in the Zigbee OTA Upgrade process:

- OTA Upgrade Client – Runs on the target, that is the device that is being upgraded, and is responsible for downloading and installing the new firmware.
- OTA Upgrade Server – Provides the firmware image. The OTA Upgrade Server can be either a standalone third-party device or it can be instantiated on an nRF52840 DK using nRF Util's [DFU over Zigbee](#) procedure.

OTA upgrade process

The OTA Upgrade Client queries for OTA Upgrade Servers with the intervals defined by the `CONFIG_ZIGBEE_FOTA_SERVER_DISCOVERY_INTERVAL_HRS` Kconfig option until at least one server is found. Once found, the client starts to query the server for images. The interval between queries for the available Zigbee FOTA images is defined by the `CONFIG_ZIGBEE_FOTA_IMAGE_QUERY_INTERVAL_MIN` Kconfig option. After querying the OTA Upgrade Server for available images and receiving information about the image, the library begins the OTA upgrade process. The library uses [ZBOSS Zigbee stack](#)'s ZCL API to download the image.

After the OTA Upgrade Client downloads the Zigbee OTA image header, the stack verifies the following mandatory fields:

- Manufacturer ID - Defined by the `CONFIG_ZIGBEE_FOTA_MANUFACTURER_ID` Kconfig option.
- Image type - Defined by the `CONFIG_ZIGBEE_FOTA_IMAGE_TYPE` Kconfig option; it may be different than the MCUboot image type value.
- Hardware version - Defined by the `CONFIG_ZIGBEE_FOTA_HW_VERSION` Kconfig option.
- Firmware version - Defined by the `CONFIG_MCUBOOT_IMGTOOL_SIGN_VERSION` Kconfig option; see [Configuring Zigbee FOTA](#) for details.

If all values are accepted, the OTA Upgrade Client downloads the first fragment of the firmware image.

Once the download is started, all received data fragments are passed to the [DFU multi-image](#) library. The library takes care of where the upgrade candidate is stored, depending on the image type that is being downloaded.

When the download is completed, the download client sends an appropriate event. At this point, the received firmware is tagged as an upgrade candidate and the OTA server is queried for an update time.

Once the OTA server triggers the update process, the library sends a `ZIGBEE_FOTA_EVT_FINISHED` callback event. When the consumer of the library receives this event, it should issue a reboot command to apply the upgrade.

Configuration

To enable the Zigbee FOTA library, set the `CONFIG_ZIGBEE_FOTA` Kconfig option.

To configure the Zigbee FOTA library, use the following options:

- `CONFIG_ZIGBEE_FOTA_HW_VERSION`
- `CONFIG_ZIGBEE_FOTA_DATA_BLOCK_SIZE`
- `CONFIG_ZIGBEE_FOTA_ENDPOINT`
- `CONFIG_ZIGBEE_FOTA_PROGRESS_EVT`
- `CONFIG_ZIGBEE_FOTA_MANUFACTURER_ID`
- `CONFIG_ZIGBEE_FOTA_IMAGE_TYPE`
- `CONFIG_ZIGBEE_FOTA_COMMENT`
- `CONFIG_ENABLE_ZIGBEE_FOTA_MIN_HW_VERSION`
- `CONFIG_ZIGBEE_FOTA_MIN_HW_VERSION`
- `CONFIG_ENABLE_ZIGBEE_FOTA_MAX_HW_VERSION`
- `CONFIG_ZIGBEE_FOTA_MAX_HW_VERSION`

- `CONFIG_ZIGBEE_FOTA_SERVER_DISCOVERY_INTERVAL_HRS`
- `CONFIG_ZIGBEE_FOTA_IMAGE_QUERY_INTERVAL_MIN`

For detailed steps about configuring the library in a Zigbee sample or application, see [Configuring Zigbee FOTA](#).

Limitations

The Zigbee FOTA library has the following limitations:

- The endpoint definition in the library includes the endpoint ID, defined with `CONFIG_ZIGBEE_FOTA_ENDPOINT`. When using the Zigbee FOTA library, this endpoint ID cannot be used for other endpoints.
- The Zigbee FOTA upgrades are currently only supported on the nRF52840 DK (PCA10056) and nRF5340 DK (PCA10095).
- The Zigbee FOTA library does not currently support bootloader upgrades.

Additionally, the following limitations apply on the nRF5340 SoCs:

- It is required to use external flash to enable the Zigbee FOTA library.
- By default, only the full upgrades (to both application and network core) are allowed. Disable the `CONFIG_NRF53_ENFORCE_IMAGE_VERSION_EQUALITY` Kconfig option to build update images without inter-dependencies so that they can be applied independently.
- It is impossible to enable `SB_CONFIG_MCUBOOT_MODE_SWAP_WITHOUT_SCRATCH` Kconfig option. As a result, the fallback recovery is not available and any valid upgrade will overwrite the previous image. The call to the `boot_write_img_confirmed()` will have no effect.
- The current DFU limitations and dependencies are enforced by the `CONFIG_NRF53_MULTI_IMAGE_UPDATE` Kconfig option.
- The version of the network core image is always set to the same value as the application core image. Its value can be configured using the `CONFIG_MCUBOOT_IMGTOOL_SIGN_VERSION` Kconfig option.
- The MCUboot header is not stored inside the network core flash memory. As a result, it is impossible to read the version of the currently running network core image.

API documentation

Header file: `include/zigbee/zigbee_fota.h`

Source files: `subsys/zigbee/lib/zigbee_fota/src/`

[Zigbee firmware over-the-air download library](#)